ETX on MI300X

Qwen3-30B-A3B bf16 decode · batch 1 · 2026-10-01

Result

Configuration	ms/token	vs vLLM
vLLM 0.27.1 default	4.862	—
vLLM + AITER	4.975	+2.3%
ETX megakernel	**4.865**	**+0.06%**
ETX unfused — same tiles, HIP-graph replay	4.998	+2.8%

> **ETX reaches parity with vLLM, and fusion pays.** 4.865 against 4.862 — 0.003 ms, inside the 0.017 run-to-run spread, so the two are indistinguishable. A persistent kernel beats a HIP-graph replay of the same tiles; the day began with fusion *losing* by 4.2%. The system improved 17.5% against itself.

Wall clock, the basis vLLM's benchmark uses. Three interleaved runs per row, one binary generation, all 32/32 tokens against HuggingFace greedy with every DSL operator of the last layer within 2 bf16 ulps of a CPU reference. Machine: 1× MI300X VF, ROCm 7.14. Confirmed on bare metal (ROCm 7.2.4) before the last change: 5.042 / 5.266 / 4.887, fusion margin +0.224 against −0.235 on 2026-09-30.

Why fusion now pays

Six changes account for the 17.5%. The first is the whole story; the rest are the tail.

Change	ms	Mechanism
Data-class coherence	**−0.317**	Every task's acquire was `buffer_inv sc1` — a whole-L2 invalidate — so each phase began cold. DSL tiles now read activations coherently and skip the acquire; weights stay cached.
Unfolding the MoE top-k	**−0.231**	Our own earlier "optimisation": 448 expert tasks per layer each redid a 128-logit softmax on wave 0 while 15 waves waited. Ablation priced it at 0.676.
Async per-step host work	**−0.152**	20 synchronous HIP calls per step, a dead 241 KB memset, a 50 μs poll sleep.
Non-temporal weight loads	**−0.106**	6.08 GB read once per token were evicting the MALL-resident KV cache. L2 hit 31.9 → 34.6%.
Vectorising the coherent path in `attnprep`	**−0.074**	The same disease, in the tile where it was worst: 3400 coherent requests per task became 768. The new lane→element mapping puts the neox rope partner in the same lane's registers, so the shuffle disappears, and register spills fell 16 → 8 — a whole-kernel effect that also moved `attn` and `o_proj`.
Vectorising the coherent norm path	**−0.105**	A coherent access bypasses L1 and L2, so a 2-byte accessor issues one memory request per lane per element: the folded norm staged 4096 requests per task against `o_proj`'s 1024 at four times the width. 16-byte accesses cut `qkv`'s body 11.81 → 9.50 μs. Predicted not to move the MoE phases, which already use the wide path; it did not.
Phase width, import prune	**−0.082**	304 → 256 tasks; an unused imported tile was setting the LDS arena.

The invalidate is the one mechanism worth stating twice, because everything rests on it. An L2-resident phase costs 0.76 μs without it and 3.14 μs with it — **4.1×**. A dependent load chain crossing it pays 1107 ns per hop against 103 ns — **10.6×**. Both figures reproduce on both machines.

What ETX is actually worth

Against the same model with *all* kernels imported from vLLM instead of our DSL tiles — one build, one machine, same day:

Build	fused	unfused	fusion margin
All imported vLLM kernels	6.435	6.264	**−0.171 — fusion loses**
Imported + vLLM's fused q/k-norm+rope	6.071	5.787	**−0.284 — fusion loses**
Our DSL tiles	**4.937**	**4.998**	**+0.061 — fusion wins**

Pairing every phase and its folds, the 1.50 ms difference decomposes exactly (Δ sums to 31.3 μs/layer × 48 = 1.50 against a measured 1.497):

Source of the difference	ms
`prep` as 4 tasks against a one-workgroup q/k-norm+rope+cache chain (16.24 μs)	**+0.614**
Attention placed by KV head — *the same imported kernel*, 20 tasks against 5	**+0.355**
Folds deleting `silu_and_mul`, `moe_sum`, `post_norm` and their hand-offs	**+0.500**
The acquire and the remainder	**+0.030**
Kernel speed, our tiles against vLLM's, summed like-for-like	**+0.043 — a tie**

> **The two tile sets are equally fast; everything ETX gains is structural.** Our GEMV beats `wvsplitk` on the same 5120×2048 shape with fewer workgroups (6.29 against 6.75) and loses by similar margins elsewhere — net 0.043 ms, inside the noise. What cannot be done with imported kernels is restructuring: an imported kernel dictates its own launch shape, and its loads come from vLLM's source, so its task must keep the `buffer_inv sc1` acquire that invalidates L2 for everything after it. **ETX's tiles earn their place by being editable, not by being faster.**

Why ETX is still behind

Where the time goes, from a decode trace and hardware counters taken before the last change (which removed 0.105 of the tile item):

Component	ms	Owner
Streaming weights at the achievable rate	1.594	hardware
Workgroup start delay	0.414	hardware — ETX's spread is below the dispatcher's own
Attention kernel fixed cost	0.301	imported vLLM kernel, already above the HBM rate
Hand-off, 9 edges × 1.00 μs	0.433	ETX — at the protocol floor
Per-step host work	0.135	ETX
Our own DSL tiles	**1.29**	**prep+reduce 0.548 · MoE 0.476 · top-k 0.391, less the folded norm now recovered**

ETX's orchestration is spent; the tiles are not. Hand-off sits at its measured floor, start delay is below hardware jitter, host work is 0.135 ms, and the instruction cache — named in the literature as a megakernel's weakness — hits 99.90%. The gap to vLLM is 0.075 ms; the tile item alone is 1.29. Kernel quality was never ETX's claim — and the phase-by-phase pairing shows our tiles tie vLLM's on speed. The residual is distance from the *hardware* rate, which vLLM's kernels are equally far from; matching them everywhere would gain 0.119 ms, barely passing vLLM, and is a waypoint rather than a ceiling.

One hardware limit bounds the tiles: occupancy is **4.000 of 8 waves per SIMD**, capped by 128 VGPRs, with **85.9% of wave-cycles waiting** and zero memory-unit stalls. Latency is unhidden; bandwidth is not the constraint. Reaching 6 waves needs VGPR ≤ 85 and LDS ≤ 21845 B together — a tile redesign, not a flag. Smaller workgroups do nothing: the register term is 4 at any size.

Batch: the premise was wrong

Batch	vLLM	ETX	Ratio
1	4.861	5.267	1.08×
2	2.793	3.598	1.29×
4	1.820	3.466	**1.90×**

ETX replicates where vLLM batches. Task counts are exactly linear in request count — 59,978 / 119,956 / 239,912 — so every request streams its own copy of the weights, while vLLM reads a weight tile once per B-column activation matrix. Batch is ETX's worst regime, not its best.

The remedy was built three times, and each build refuted its own diagnosis. Dense-only batching lost by 0.25 ms and was blamed on the second column doubling the arithmetic — refuted by counting it (FMA is 0.7% of the regression, so matrix instructions were rejected without being built). The blame moved to the per-phase joins — refuted by removing them: whole-layer batching with an on-device expert union brings nine joins per layer down to one and loses by **more** (7.48 against 6.90 replicated per two-token step), because the join was cheap all along. The real costs, from the traces: the batched MoE runs 384 tasks over 304 workers, so a quarter of them queue as a second serial wave; and lockstep batching destroys the ∼12 μs stagger replication gets free, in which each request fills the other's stalls. **Batching cannot beat replication under a lockstep phase model.** Sharing weights while keeping the stagger needs split-phase scheduling the runtime does not have — that is the wall, stated rather than attempted. ETX at B≥2 ships replicated.

What vLLM's fusion is worth, for scale

vLLM configuration	ms/token
Default — compiled, fused, full graph replay	4.862
Without Inductor operator fusion	4.889
Graphs disabled	32.710
Eager	35.134

vLLM's graph is worth **6.7×**; its operator fusion 0.6%. Host dispatch dominates everything, and both systems have already removed it — vLLM by graph capture, ETX by the persistent kernel. On top of that, ETX's megakernel adds 2.0% and vLLM's operator fusion 0.6%: the same order, both an order of magnitude below what avoiding per-operator dispatch buys.

Ruled out, with numbers

Idea	Result	Reason
Three barriers per task → two	**+0.095**	Holding the next descriptor live cost a tile-sized LDS allocation.
16-byte load path for `moe_down`	**+0.144**	Address tracking and a second wave reduction exceed the wider loads.
Attention block over 8 XCDs	+0.008	Latency-bound: `prep` moves 2.5 KB per task in a 5.9 μs body.
Hoisting `prep`'s scalar loads	+0.003	The compiler already issued them as early as dependences allow.
Batching the top-k stores	+0.008	Its body is one write-through round trip, not 24 separable costs.
Next-task argument prefetch	−0.003	The argument path was not the exposed cost.
HIP graph over the per-step host sequence	0.000	Nothing left to collapse once the calls are async.
Device-side per-step reset	−0.008	Works, 0.16%; not worth a reset path, an exit counter, two fences and a guard.
Blocking instead of polling	−0.002	The 71 μs is launch-to-start latency, not the wait strategy.
Smaller workgroups	not built	Register term caps waves/SIMD at 4 at any size.
Fusing `prep` into `attn`	not built	`prep` is in-place: more than one task per KV head applies norm and rope twice.
Narrowing `reduce`'s acquire	not built	`buffer_inv sc0` leaves stale L1 copies; uncached loads cost the same 690 ns.

Three measurement errors, ours

1. **Traces taken during the prompt.** Limited to the first eight steps — the teacher-forced prompt, sequence length 1–8. vLLM's attention kernel partitions by a hardcoded 256 tokens and exits past the end, so 16 of 20 tasks looked trivial and we concluded a 4.3× work imbalance. In decode the spread is 1.17×. Timing was unaffected; the attention block was understated by 0.21 ms.
2. **A statistic that charged imbalance to skew.** In-phase skew was computed as span minus *mean* body, but a phase ends when its last task does. Recomputed: 0.414 start delay, 0.244 imbalance.

3. **The baseline measured at the wrong width.** The streaming benchmark used 256-thread workgroups; the tiles use 1024. Corrected, the MoE residual falls 0.532 → 0.476 and the router's 0.164 → 0.046 — most of what we had charged to ETX there was the hardware floor.

What is left

Item	ms	What it needs
`prep` + `reduce`	0.548	The same disease the folded norm had, and the next thing to fix: 2-byte per-lane coherent accessors turn a 256-byte head row into 128 requests, ~2300 per task. Widening them here means re-deriving the lane→element mapping the rope shuffle (`lane ^ 32`) and the 2-elements-per-lane KV writes depend on — not a widened accessor.
MoE tile bodies	0.476	~70% of achievable. Bytes-in-flight and the pass structure are excluded by arithmetic; the gather is 7%, the epilogue 5%; the request-width explanation was tested and does not apply — the MoE path is already 8 bytes per request. No mechanism identified.
Top-k tile	0.391	At its floor: one write-through round trip behind which every store sits. The only lever reintroduces the invalidate.
Register pressure	unknown	128 → 85 VGPRs would take occupancy to 6 waves/SIMD against 85.9% wave-cycles waiting. Bound unmeasurable until built.
Batching	—	Matrix instructions plus whole-layer batching with no per-phase join.

The compiler and runtime half of this project is at its measured floor. What separates ETX from vLLM at batch 1 is hand-written tiles against kernels AMD has tuned — worth saying plainly, because it is not what ETX set out to prove, and because what it did set out to prove is now measured on two machines against the strong baseline.

---

32 tokens per run verified against HuggingFace greedy and a CPU reference for every DSL operator of the last layer. Primary: 1× MI300X VF, ROCm 7.14. Validation: 1× MI300X bare metal, ROCm 7.2.4. vLLM 0.27.1. Absolute milliseconds are not interchangeable between the two machines; ratios and the fusion margin are. Logs, traces and profiler output are archived with checksums in `docs/DATA.md`; the per-phase budget in `docs/BUDGET-2026-10-01.md`.